G01 | The assessment map

CS1101S • Source Academy Python §3 • Programming Assessment 2 preparation

This guide develops reusable reasoning from the nine Practice A questions and the two syllabus screenshots. It does not predict Practice B. The worked code uses the pair/head/tail/None convention visible in your questions, with indexed lists for matrices and stacks. Every executable example targets Source Academy Python §3, using its documented grammar and predeclared primitives. The browser dojo supplies any additional helpers explicitly. Later sections deliberately change constraints and representations so the same ideas can transfer.

What Practice A actually demonstrates

Question	Marks	Underlying technique	Contract that matters
Q1 route distance	5	Adjacent pairs + aggregation	Directed matrix: mat[i][j]
Q2 delta encoding	5	Previous-value state	Preserve input
Q3 delta decoding	5	Running-total state	Preserve input
Q4 alternating columns	8	Index formula + parity	Independent rows
Q5 postfix	8	Stack + operand order	Right operand first; track stack top
Q6 shortest path	10	Recursive search + undo	Restore the maze
Q7 characters	5	Delayed tail + index state	O(1) construction
Q8 Kolakoski	10	Run state + self-reference + caching	Fast first 1000 elements
Q9 destructive merge	8	Reuse nodes + redirect links	No new pairs

These questions total 64 marks. The marks describe this practice only; they are not a forecast of the real assessment's distribution.

Three levels of coverage

Direct evidence: lists, matrices, stacks, backtracking, streams, memoization and destructive updates are explicitly exercised. These receive the deepest treatment here.

Supporting foundations: expressions, scope, recursion, iterative processes, higher-order functions, data abstraction, testing and orders of growth explain why the solutions work.

Syllabus extensions: trees, CPS, symbolic processing, searching, sorting, environments, runes, curves, sound and the metacircular evaluator are named in the screenshots. They receive a transfer-oriented introduction. The screenshots do not supply their detailed APIs or the exact evaluator representation, so this guide does not invent those details.

Guide map

G02-G08: contracts, Python §3, references, recursion and lists. G09-G13: matrices, stacks and search. G14-G18: streams and Kolakoski. G19-G21: mutation, sorting and complexity. G22-G24: trees, CPS, evaluators and wider syllabus. G25-G26: testing and independent practice.

The dojo has 36 original problems. The solution book is separate so the guide and two-sided reference sheet can support genuine attempts.

G02 | Read the contract before choosing an algorithm

A function specification describes more than its numerical output. It defines legal inputs, representation, allowed changes, resource requirements and sometimes identity. A program can print the right numbers and still violate the question.

A six-part contract

1. **Input:** type, shape and guarantees. Is it an indexed list, linked list, pair record or stream? Is it empty, finite, sorted, rectangular or disjoint?

2. **Output:** number, Boolean, new structure, existing node, or delayed computation. Does the first element have special treatment?

3. **State:** the minimum information from earlier work needed to process the rest.

4. **Mutation:** forbidden, permitted temporarily with restoration, or required. Are heads allowed to change?

5. **Cost:** initial call, one step, entire finite input or first n stream values? Count the correct operation.

6. **Boundary:** empty, singleton, last element, no solution, duplicates, ties, start at goal.

Vocabulary that the code relies on

A **node** is one linked pair. A **prefix** is the initial portion; a **suffix** is what remains after a prefix. An **invariant** is a statement true at each loop/helper boundary. **Reachable** means there exists a sequence of permitted moves from one position to another. **Arity** is the number of inputs an operator consumes. **Nullary** means taking zero arguments. **Productive** means a stream can supply the next requested value after finite work. **Stable sorting** preserves the original order of equal-key items.

Core Source Academy primitives

Operation	Meaning in these materials
pair(x,y)	Construct a fresh two-field pair; y need not be a list
head(p), tail(p)	Read the pair's two fields
llist(a,b,c)	Shorthand for pair(a,pair(b,pair(c,None)))
is_none(xs)	Test the linked-list/stream terminator
set_tail(p,q)	Change an existing pair's tail
stream_tail(s)	Force the function stored in tail(s)
memo_fun(f)	Wrap a nullary function so its successful result is cached
math_inf	Positive infinity: larger than finite path lengths

All listed names except memo_fun are predeclared in the documented Python §3 library. memo_fun is a lecture-style helper: its complete definition is in G16 and included in dojo programs that need it. stream(a,b,c) constructs a finite stream. len(a) is for indexed lists/strings; length(xs) traverses a linked list. append(xs,ys) concatenates linked lists; it is not an indexed-list push. Always respect any additional restrictions in the assessment.

A reliable construction sequence: write a tiny example; define the helper's meaning in one sentence; settle the base case; do one unit of work; make progress; combine or return; verify the contract. A function name or similar story is weaker evidence than these details.

G03 | Python §3: the language you will actually use

Source Academy Python §3 is a teaching language with Python-like syntax. The distinction matters: code accepted by an ordinary Python interpreter can still be outside this language. The website's code targets the documented 2026 §3 grammar. Use the official Source Academy playground with Python §3 selected to execute it.

A small, dependable working vocabulary

Use one-name assignment, def, return, lambda, if/elif/else, while, for i in range(...), break, continue, list literals and indexed access/assignment. Use pair/head/tail for linked structures. nonlocal is available when a nested function rebinds an enclosing local variable. An assignment within a for/if block does not create a new block scope.

Full-Python habit to avoid here	Write this instead
x += 1	x = x + 1
a, b = x, y	Separate assignments; save old values if swapping
for x in items	for i in range(len(items)), then x = items[i]
a.append(x), a.pop()	Preallocate capacity and track a used length
a[1:], comprehensions, tuples	Indices, explicit loops, list records or pairs
0 <= i < n	0 <= i and i < n
def f(x=0)	Explicit parameter; call f(0)
if xs or if not number	Explicit Boolean test such as is_none(xs) or number == 0

Membership tests, attribute methods, dictionaries, classes, exception statements and f-strings are also outside the grammar used by this pack. The examples do not need them.

Boolean tests and evaluation order

Conditions must be Boolean. A number or list is not a substitute for True/False. and/or short-circuit, so a guard can prevent an invalid selector:

```
if is_none(xs) or is_none(tail(xs)):
    return None
```

The left test is evaluated first. If it is True, tail(xs) is never attempted. Write bounds checks before indexing for the same reason. Use == for values and is for object identity; do not use is for numeric equality.

Calls, return and numbers

f names a function; f() calls it; lambda: f() postpones that call. A def needs return to deliver a result: returning inside a helper does not return from the outer function. Print the result you want to inspect with print(...).

/ divides; // floors; ** raises to a power. For example, -3 // 2 is -2. Parentheses make precedence explicit. range(a,b) excludes b. Update loop state so the stopping condition eventually holds.

§3 supports proper tail calls: a call returned directly can use constant control-stack space. A recursive call inside an addition or pair construction is not in tail position. G05 and G22 distinguish control space from retained data.

Reference: Source Academy Python §3 specification and predeclared-library documentation. This is a practical vocabulary, not a replacement for the full language reference.

G04 | Names, aliases, identity and copies

Treat a variable as a name referring to an object. Assignment binds a name; it does not automatically copy the object. If two names refer to one mutable object, mutation through either is visible through both. This matters for indexed lists, matrix rows and linked pairs.

```
a = [1, 2]
b = a
b[0] = 9          # a is now [9, 2]
b = [7, 8]       # rebind b; a is still [9, 2]
```

In Python §3, == compares lists/pairs structurally, while is checks identity. Separately constructed equal lists are not the same object. Use is when the question asks whether a returned pair is one of the original nodes. Restrict structural-equality reasoning to the finite acyclic structures intended here; identity is the right test for a cached stream suffix.

Shallow copying

```
row = [1, 2]
a = [row, row]
b = [a[0], a[1]] # new outer list; same two inner references
b[0][1] = 9      # both a rows and both b rows show the change
```

An outer copy does not copy nested objects. To copy a numeric matrix, allocate an outer list and a fresh row for each original row, then copy each entry with nested range loops. That isolates later numeric-cell updates. If cells themselves contain mutable objects, another layer may still be shared.

The repeated-row trap

```
bad = [[0] * 3] * 2
bad[0][1] = 7    # BOTH rows now contain 7

good = [None] * 2
for r in range(2):
    good[r] = [0] * 3     # a fresh row each iteration
```

Repeating references to immutable zeros is harmless; repeating a reference to one mutable row is not. The same issue applies to visited tables with another nested dimension.

Linked-list aliasing

copy = xs creates an alias, not a linked-list copy. But moving a local cursor with copy = tail(copy) is only rebinding; it does not change xs. set_tail(copy, other) changes an actual node. Nonmutating construction pair(head(xs), transformed_tail) makes a fresh outer node, while its head value may still be a shared mutable object. "Does not modify the input" and "returns a fully independent deep copy" are different guarantees.

A destructive algorithm must preserve the required node identities, not merely numerical equality. Conversely, nonmutating algorithms must leave every original head and tail unchanged, even if their returned values look correct.

Sources: Practice A Q2-Q3, Q4, Q9; Source Academy Python §3 list and identity semantics.

G05 | Recursion, iteration and a precise helper meaning

A recursive function solves an instance using smaller instances of the same contract. Three ingredients make the argument work: a correct base case, a progress measure that decreases, and a combination step that turns correct smaller results into the desired larger result.

```
def total(xs):
    if is_none(xs):
        return 0
    return head(xs) + total(tail(xs))
```

The contract is "return the sum of this suffix." The empty suffix sums to 0. tail(xs) has one fewer node. Assuming the recursive call sums that suffix correctly, adding the first element gives the whole sum. This assumption is how you design recursion; mentally expanding every call is not necessary to justify it.

What remains pending

For total(llist(2,3,4)), the first frame waits for 2 + (...), the next for 3 + (...), and the next for 4 + 0. Those pending additions are stored in call frames. The runtime stack grows linearly even though the final answer is a single number.

```
def total_loop(xs):
    answer = 0
    while not is_none(xs):
        answer = answer + head(xs)
        xs = tail(xs)
    return answer
```

Here answer is the sum of the processed prefix and xs is the unprocessed suffix. The loop maintains that meaning after every step. No pending additions remain outside the explicit state.

Tail position and proper tail calls in §3

```
def total_state(xs, answer):
    if is_none(xs):
        return answer
    return total_state(tail(xs), answer + head(xs))
```

The recursive call is the entire returned result: a tail call. At the process-model level, the state is a constant-size summary. Python §3 executes proper tail calls in constant control-stack space. Thus total_state uses O(1) auxiliary control space under the unit-cost model, just like the loop. In total above, addition remains pending, so that recursion needs O(n) control space. An iterative process is characterized by bounded explicit state, not simply by whether def calls itself.

Base cases come from the operation

An elementwise map needs zero elements to stop. An adjacent-pair operation stops with fewer than two. A merge stops when either input is empty. A divide-and-conquer sort stops at length 0 or 1. A grid search stops on invalid moves or success. A stream usually constructs one node now and makes future progress only when forced.

Write a progress measure before debugging: list length, unprocessed index range, remaining repetitions, moves remaining, or unvisited states on this path. If no measure decreases, suspect a cycle or an unchanged recursive argument.

Source focus: Practice A Q1-Q3, Q6-Q9; syllabus recursion and iterative processes.

G06 | Adjacent pairs, accumulators and scans

Three superficially similar traversals require different state.

Task	What one output/action needs	State to carry
Adjacent difference	Current and next input	Cursor or previous input
Delta encoding	Current input minus previous input	Previous ORIGINAL value
Delta decoding	Current difference plus previous decoded value	Running decoded total

Directed route aggregation

```
def route_distance(mat, route):
    if is_none(route) or is_none(tail(route)):
        return 0
    a = head(route)
    b = head(tail(route))
    return mat[a][b] + route_distance(mat, tail(route))
```

A route of n houses has n-1 legs. Adjacent legs overlap at a house, so advance by one node. The matrix is directed: mat[a][b] can differ from mat[b][a]. A maximum-leg variation changes + to a maximum aggregation; it does not change the traversal.

Delta encoding and decoding

```
def delta_encode(xs):
    def walk(rest, previous):
        if is_none(rest): return None
        current = head(rest)
        return pair(current - previous, walk(tail(rest), current))
    if is_none(xs): return None
    return pair(head(xs), walk(tail(xs), head(xs)))

def delta_decode(ds):
    def walk(rest, total):
        if is_none(rest): return None
        updated = total + head(rest)
        return pair(updated, walk(tail(rest), updated))
    return walk(ds, 0)
```

For [7,10,6], encoding emits [7,3,-4]. The encoder passes 10, not 3, as the next previous value. For differences [7,3,-4], decoding carries totals 7,10,6. The accumulator means the decoded value BEFORE consuming the next difference. Naming the new value updated avoids confusing the old and new state.

A **fold/reduction** returns one aggregate. A **scan** returns all successive aggregates. Delta decoding is a sum scan. An initial balance can be carried without being emitted as an extra output. Check whether a specification wants n outputs or n+1; both are legitimate scan conventions.

Source focus: Practice A Q1-Q3. Each implementation is O(n) time, O(n) recursive stack; encoding/decoding additionally allocate O(n) output pairs.

G07 | Higher-order functions, closures and scope

Functions are values: they can be arguments, results and stored data. A higher-order function consumes or produces a function. A lambda is an expression that creates a function; its body is evaluated only when called.

```
def compose(f, g):
    return lambda x: f(g(x))

shift_then_square = compose(lambda x: x*x, lambda x: x+2)
# shift_then_square(3) == 25
```

The order matters: f(g(x)) applies g first. The returned function remembers the bindings of f and g in the call to compose. This combination of a function and its enclosing environment is a **closure**. The environment outlives the factory call when a returned function still needs it.

Local names and nonlocal

```
def make_counter(start):
    value = start
    def add(amount):
        nonlocal value
        value = value + amount
        return value
    return add
```

Assignment to a name in a function makes it local unless declared otherwise. nonlocal tells Python to rebind an existing name in an enclosing function scope. Reading that name does not need nonlocal. Mutating an enclosing list entry, such as history[0] = x, does not rebind the history name and does not need nonlocal. In §3, nonlocal itself is supported. Explicit helper parameters are also useful when the state naturally progresses from one call to the next.

Separate calls to make_counter create independent environments. Two names bound to the same returned function share its state. This distinction is the function-object version of list aliasing.

Map, filter and right fold

```
def map_ll(f, xs):
    if is_none(xs): return None
    return pair(f(head(xs)), map_ll(f, tail(xs)))

def fold_right(op, initial, xs):
    if is_none(xs): return initial
    return op(head(xs), fold_right(op, initial, tail(xs)))
```

fold_right(op,z,llist(a,b,c)) means op(a,op(b,op(c,z))). Operand order matters for subtraction, division and construction. A left fold instead repeatedly updates acc = op(acc,current). They agree for some associative operations with suitable identities, not universally. For subtraction on [1,2,3] with 0, the right fold is 2 and the left fold is -6.

A loop-created lambda normally reads the captured variable when called, not a frozen snapshot from creation. Prefer a factory/helper parameter to capture a per-step value. This is especially important when deferred stream tails use mutable loop variables.

Sources: syllabus higher-order functions, scope and environment model; Source Academy Python §3 scope rules. These explicit map_ll/fold_right definitions remove ambiguity about operand order; the predeclared linked-list map/filter/reduce have their documented contracts.

G08 | Linked-list construction and data abstraction

A pair is more general than a linked list. pair(4,2) is a record with two fields. llist(4,2) is two linked nodes ending in None. They are different shapes and require different selectors. For the record, tail(record) is 2; for the list, tail(xs) is llist(2).

Respect the representation boundary

Use pair, head and tail rather than relying on how the library implements them. An abstract pair could be a Python list, an object or a function. If you index a linked node using xs[0], your code depends on implementation details absent from the problem. A selector states what data you mean, while a representation-specific index states how it happens to be stored.

The standard fresh-output pattern

```
def keep_positive(xs):
    if is_none(xs): return None
    rest = keep_positive(tail(xs))
    if head(xs) > 0:
        return pair(head(xs), rest)
    return rest
```

The suffix is transformed once. Prepending the current item to an already correctly ordered suffix preserves order. If rejected, returning rest skips that item without altering the input. This is an ordinary filter, not a destructive filter.

Prepending reverses order in an iterative builder

```
answer = None
while not is_none(xs):
    answer = pair(head(xs), answer)
    xs = tail(xs)
```

The newest value becomes the first, so this returns the reverse. To preserve order, use recursive construction; build a reversed accumulator then reverse; or maintain an end pointer if mutation of newly created output nodes is allowed. Each choice has different stack and mutation implications.

Why repeated append can become quadratic

For linked lists, appending a singleton to the end often walks or copies the entire existing prefix. Building lengths 1,2,...,n that way costs 1+2+...+n = Theta(n^2). Constant-time prepend plus one reversal is linear. In §3, append(xs,ys) means linked-list concatenation. Indexed-list stacks instead use a preallocated list and a top index. Do not transfer operations or costs between these representations.

A nested linked list is still a linked list whose head values may themselves be lists. A proper list ends in None; a record may have a non-list tail. A function that traverses all tails as though they were list links is wrong for arbitrary pair records. A constructor/selector contract removes that ambiguity.

Source focus: Practice A Q1-Q3, Q8-Q9; syllabus data abstraction and list processing.

G09 | Matrices: distinguish coordinates from traversal order

An indexed-list matrix mat is an outer list of rows. mat[r][c] first selects row r, then column c within it. It is not mat[x][y] unless x explicitly means row. With R rows and C columns, valid coordinates satisfy 0 <= r and r < R and 0 <= c and c < C.

Derive the alternating-column formula

Column c contains the c-th block of R numbers, so its first numerical value is c*R+1. On even columns, the offset from that value is r. On odd columns, it is R-1-r. Therefore:

```
value = c * R + 1 + (r if c % 2 == 0 else R - 1 - r)
```

For R=4, column 1 contains 8,7,6,5 from top to bottom. At r=0, the offset is 3 and the value is 1*4+1+3 = 8. At r=3 it is 5. Checking the two ends is an efficient way to catch an off-by-one error.

```
def alt_column_matrix(R, C):
    mat = [None] * R
    for r in range(R):
        mat[r] = [0] * C
        for c in range(C):
            offset = r if c % 2 == 0 else R - 1 - r
            mat[r][c] = c * R + offset + 1
    return mat
```

We construct rows because that matches the output representation, even though the numbering story fills columns. A formula frees you from following the story's traversal order literally. An alternative is to allocate independent rows of zeros and then walk each column in its required direction.

A general indexing recipe

Separate **block offset**, **position within a block** and **direction**. For an alternating-row variant the block size is C, not R. For transpose, output[c][r] = input[r][c], and dimensions swap. For a column sum, hold c fixed while r changes. For a row sum, hold r fixed while c changes.

Use nonsquare examples: a 2-by-4 test detects swaps that a 3-by-3 matrix can hide. Also test 1-by-C and R-by-1. Check row independence with a mutation to one cell of a newly constructed result, rather than only printing it: aliasing may be invisible until a later update.

Complexity is Theta(RC) for constructing all RC cells. Auxiliary space excluding the returned matrix is O(1) for this implementation. Source focus: Practice A Q4.

G10 | Postfix: a stack and an explicit top index

A stack follows last-in, first-out order. In §3, allocate enough indexed-list slots in advance. top is the number of live values, and stack[top-1] is the most recent value. Slots at or above top are irrelevant; popping does not need to erase them.

The invariant is: after a token prefix, the live stack contains the values of completed subexpressions that later operators have not yet consumed, in their original left-to-right order.

Token in 9 4 - 2 /	Live values after processing
9	[9]
4	[9,4]
-	[5]
2	[5,2]
/	[2.5]

```
def eval_postfix_exp(tokens):
    stack = [None] * len(tokens)
    top = 0
    for i in range(len(tokens)):
        token = tokens[i]
        if is_string(token):
            top = top - 1
            right = stack[top]
            top = top - 1
            left = stack[top]
            if token == '+':
                value = left + right
            elif token == '-':
                value = left - right
            elif token == '*':
                value = left * right
            else:
                value = left / right
            stack[top] = value
        else:
            stack[top] = token
        top = top + 1
    return stack[0]
```

The input contract guarantees valid binary operators and enough operands. Each numeric token adds one value. Each binary operator removes two and adds one, decreasing top by one overall. The first removed value is the RIGHT operand. Subtraction and division expose reversal bugs that addition hides.

If validation is requested, check the operator and available live count before accessing operands, and require top == 1 at the end. A unary negation changes the top value without changing top. Do not add validation requirements to a question that already guarantees validity.

Why a stack matches the expression grammar

Two completed postfix subexpressions followed by an operator collapse into one completed value. This is the same structure as a binary expression tree, without explicitly building the tree. Operator arity and operand order survive changes between prefix, infix and postfix notation.

Each token takes constant work: O(t) time and O(t) allocated stack slots. Source focus: Practice A Q5 and syllabus symbolic processing.

G11 | Backtracking: choose, explore, undo

Backtracking explores alternatives while recording the choices made on the current candidate path. After exploring one branch, it reverses temporary changes so another branch starts from the correct state.

For the maze, a call means "find a result from this cell, given the cells already used on the current path." A rectangular grid is a graph: traversable cells are vertices and permitted orthogonal moves are edges. You do not need an adjacency-list representation to reason this way.

Why marking is needed

Without a visited-path rule, a recursive search can move A to B to A to B forever. Marking the current cell as blocked prevents the next calls from stepping back into any cell already on this path. Since each step then uses a new cell, recursion depth is bounded by the number of cells.

Why restoration is equally necessary

Suppose one branch visits a junction and fails. Another branch may legitimately need that junction. If the first branch leaves it blocked, the second branch solves a different maze. The temporary marks describe one candidate path, not permanent facts about the original map.

The invariant is: **when a call finishes, all changes made by that call and its descendants have been undone.** This lets sibling calls rely on the same starting state, apart from their ancestors' still-active marks.

Choose the correct result algebra

Search objective	Invalid / blocked	At goal	Combine children
Existence	False	True	OR
Number of simple paths	0	1	Sum
Shortest number of moves	infinity	0	1 + minimum

For shortest paths, zero at the goal counts additional moves, not visited cells. Returning 1 at the goal makes every answer one too large. Infinity is an impossible-route sentinel because min(finite,infinity) selects the feasible route and 1+infinity remains infinity.

Always check coordinates before indexing. Then reject walls; then handle the goal; then mark/explore/restore. If you return directly after marking, the restoration line is unreachable. Store the result first, undo the mutation, then return. This remains true when an OR expression short-circuits after finding success.

Source focus: Practice A Q6. New examples and terminology explain its hint, not an assumed prerequisite supplied by the question.

G12 | Shortest-path recurrence and a complete implementation

Let F(r,c) denote the fewest additional moves from (r,c) to G under the current path marks. Invalid or blocked states return infinity. The goal returns 0. Otherwise the next step must go to one of four neighbors, so the answer is 1 plus the minimum of those four remaining distances.

```
def shortest_path_length(maze, r, c):
    R = len(maze)
    C = len(maze[0])
    if r < 0 or r >= R or c < 0 or c >= C:
        return math_inf
    if maze[r][c] == '#':
        return math_inf
    if maze[r][c] == 'G':
        return 0
    old = maze[r][c]
    maze[r][c] = '#'
    best = min(shortest_path_length(maze, r - 1, c),
               shortest_path_length(maze, r + 1, c),
               shortest_path_length(maze, r, c - 1),
               shortest_path_length(maze, r, c + 1))
    maze[r][c] = old
    return 1 + best
```

The current cell stays marked while all four alternatives run. Each child restores its own changes before the next child runs. Finally this call restores its own cell. Saving old rather than assuming '.' is good discipline when adapting the template to richer cell types.

A tiny trace

In a one-row maze ['.','.','G'], F(0,2)=0. From (0,1), the goal branch gives 0 and all other legal candidates are either blocked by the current path or outside the grid. Thus F(0,1)=1. The starting call returns 2. The grid is unchanged afterward. A start already on G returns before any mutation.

Why excluding repeated cells preserves a shortest solution

Every move costs one. If a candidate path repeats a cell, the segment between the two visits is a cycle. Removing that segment gives a shorter walk to the same goal. Therefore some shortest path has no repeated cells, and enumerating simple paths is sufficient. This reasoning must be reconsidered when state or movement rules change.

What this implementation costs

A path can branch repeatedly, so the number of simple paths can be exponential in the number of open cells. O(RC) is a stack-depth bound, not a runtime bound for this exhaustive backtracking. Small examples may still run quickly. Breadth-first search is an efficient alternative for unit-weight edges, explained next; the recursive version directly follows Practice A's hint.

Source focus: Practice A Q6. Test both output distance and exact maze restoration; either one alone is incomplete.

G13 | State design, BFS and sound memoization

The state must contain everything that determines future legal choices. A coordinate is enough for some problems, but not all. With a key, (r,c,has_key) matters. With one wall break, (r,c,break_used) matters. With exactly k moves left and revisits allowed, (r,c,k) matters. Treating different states as identical can discard the only valid solution.

Path-local marks are not a global cache

In backtracking, marks are undone because a sibling branch has a different path history. A computed shortest distance while some cells are blocked may change when a different ancestor path blocks different cells. Therefore caching that result using only (r,c) is generally unsound. A memoization key must describe the whole relevant subproblem; for simple-path search that can include the entire visited set.

For exactly k remaining moves with revisits allowed, the earlier route does not constrain the future. A cache indexed by (r,c,k) is sufficient, and k decreases on every recursive call. No path marks are needed. If reaching G early is allowed but the walk must END there after exactly k moves, the success test belongs at k==0, not at first arrival.

Why breadth-first search finds a unit-cost shortest path

BFS processes states in nondecreasing distance layers using a queue. When all edges cost one, the first discovery of a state is through a shortest route. A global visited set is then valid for that state representation. This is a different invariant from DFS path marking; permanent visited marks are not inherently wrong, only wrong when their meaning does not match the algorithm.

```
queue = [None] * (R * C)
queue[0] = [start_r, start_c, 0]
front = 0
back = 1
# Mark start discovered in a fresh visited table.
while front < back:
    state = queue[front]
    front = front + 1
    # state = [row, column, distance]
    # For a valid undiscovered neighbor, mark it now,
    # write queue[back] = [nr, nc, state[2] + 1],
    # then back = back + 1. Return on reaching goal.
```

This is a sketch, not a paste-ready function. Capacity R*C is sufficient because each cell is enqueued at most once. An expanded two-state grid needs capacity 2*R*C. The front/back indices avoid resizing or shifting. Marking when enqueued prevents many duplicate queue entries. For a grid, BFS is O(RC) time and space; an expanded key-state grid has twice as many states, still O(RC).

Changed costs change the algorithm

BFS minimizes the number of edges, not arbitrary positive edge weights. Backtracking can minimize weighted path cost on small inputs by adding the appropriate move cost; an efficient general weighted solution normally uses Dijkstra's algorithm. That named extension is not established as examinable by your screenshots. For preparation, the critical skill is recognizing when the old recurrence or cache key no longer expresses the new problem.

Source focus: Practice A Q6; syllabus memoization. BFS and weighted-search comparisons are supplementary algorithm explanations.

G14 | Streams: one value now, the rest later

A strict linked list stores its actual tail structure. A stream stores a function that will produce the tail structure when called. That function is often called a thunk. The head is available now; the tail computation is delayed.

```
# Strict list:
pair(1, pair(2, None))
# Stream:
pair(1, lambda: pair(2, lambda: None))
```

For a stream s, head(s) is a value, tail(s) is a function, and stream_tail(s) calls that function to obtain the next stream. head(tail(s)) tries to treat the thunk as a pair. tail(s)() and stream_tail(s) express the same forcing operation under this representation.

Delay the recursive call itself

```
def integers_from(n):
    return pair(n, lambda: integers_from(n + 1))
```

Python §3 evaluates function arguments before calling pair. Writing pair(n, integers_from(n+1)) would recursively construct the whole infinite tail immediately and never return a first node. A lambda delays evaluating its body; passing the result of a call does not.

O(1) character-stream construction

```
def char_stream(text):
    def build(i):
        if i == len(text):
            return None
        return pair(text[i], lambda: build(i + 1))
    return build(0)
```

The call builds at most one node. The original string is shared and the current index is small state. When the tail is forced, build executes for the next index. Empty strings immediately return None. Construction and each new step are O(1) under the course's constant-time length/indexing model; generating n elements still takes O(n).

Delayed is not automatically cheap

Slicing is outside the §3 grammar used here. More generally, even a delayed operation can be expensive when forced. Copying each remaining suffix would repeatedly process the same characters. An index carries only the position; it does not copy the text. Do not convert the whole string to a list before returning a stream when initial construction must be O(1).

A finite stream ends at None, just like a finite linked list. An infinite stream has no terminating None on its ordinary path. A consumer must bound how much it requests; eagerly converting an infinite stream to a full list cannot finish.

Source focus: Practice A Q7; Source Academy SICP 3.5.1 for the delayed-list idea. The code follows the documented Python §3 stream representation.

G15 | Stream transformations and the cost of forcing

Most list patterns transfer to streams when you move the recursive suffix into a thunk. The position of every stream_tail call determines when input is consumed.

```
def map_stream(f, s):
    if is_none(s): return None
    return pair(f(head(s)),
                lambda: map_stream(f, stream_tail(s)))

def running_stream(s, total):
    if is_none(s): return None
    updated = total + head(s)
    return pair(updated,
                lambda: running_stream(stream_tail(s), updated))
```

These versions are lazy but do not explicitly memoize; add memo_fun around their tail functions when sharing or repeated access requires it. The outer function must return its constructed stream. Defining newstream = pair(...) without returning it still gives None.

Lookahead and skipped input

An adjacent-difference stream needs the next source value to produce its current output. One forced source tail at construction is justified. The builtin eval_stream(s,n) requires n available elements: it is not a safe “up to n” collector for a shorter finite stream. A filter is subtler: it may have to search many source elements before finding the first match. Filtering an infinite stream with a predicate that never succeeds may never produce its first node. Laziness does not guarantee O(1) initial work for every transformation.

Interleaving two streams can alternate by returning the head of the first and delaying interleave(second,stream_tail(first)). Swapping their roles makes both sources progress. Concatenating an infinite first stream with a second one never reaches the second; interleaving solves a different contract.

Prefix traversal versus repeated indexing

```
current = s
for i in range(n):
    use(head(current))
    if i + 1 < n:
        current = stream_tail(current)
```

Here use is your own processing function, and n must not exceed a finite source's available length. Advancing one cursor walks n nodes once. By contrast, repeatedly calling stream_ref(s,i) from the original head traverses 0+1+...+(n-1) links: Theta(n^2) traversal even if every tail is memoized. Memoization caches generation, not arbitrary index lookup. Do not force one more tail after the final requested element unless the operation genuinely needs that lookahead.

If the source is stateful, forcing the same unmemoized tail twice may repeat side effects or yield inconsistent branches. A pure tail may still waste computation when forced twice. Cache a locally needed next stream in a variable; memoize persistent shared tails when required. Measure construction time separately from the time to generate a prefix.

Source focus: Practice A Q7-Q8; syllabus Streams I/II. Source Academy SICP 3.5.1 explains deferred computation and caching.

G16 | Memoized thunks and productive self-reference

Memoization remembers a computed result. For a stream tail, a nullary wrapper can return the same computed suffix on later calls.

```
def memo_fun(f):
    done = False
    value = None
    def force():
        nonlocal done, value
        if not done:
            value = f()
            done = True
        return value
    return force
```

The done flag is separate because None is a valid computed tail. The cache is marked complete only after f succeeds. memo_fun is not in the published §3 predeclared list. Include this definition, or use a supplied lecture version when the exercise explicitly provides one. It caches one thunk's result, not a global mapping from all possible arguments.

```
def counted_from(n):
    return pair(n, memo_fun(lambda: counted_from(n + 1)))
```

Each tail now computes once and preserves the identity of its result. Retaining the original head retains the memoized suffixes, so producing n elements generally retains O(n) space. Faster repeated access is a time-space tradeoff, not free memory.

Why a stream may refer to itself

```
ones = pair(1, memo_fun(lambda: ones))
```

The lambda is created before ones is bound, but its body is not executed until afterward. The first head already exists. Forcing the tail returns that same stream, so every head is 1. A cyclic infinite stream like this is valid; using a finite-list traversal on it would never finish.

A productive recursive definition exposes enough data before demanding itself. An unproductive definition whose first tail demands that exact same unfinished tail can recurse forever. Memoization does not fix a circular dependency that has no available seed. The seed and the dependency order must make sense first.

Sharing numerical recurrence state

A Fibonacci stream can carry two consecutive values (a,b), emit a and delay the next state (b,a+b). This explicit-state version explains the recurrence without requiring self-reference. A self-referential formulation can also work, but shared substreams must be cached to avoid repeatedly rebuilding earlier values. Prefer the form whose invariant you can state clearly.

In a function factory, a new call should create independent stream state unless sharing is explicitly requested. Global state can make one invocation interfere with another. The Kolakoski implementation next keeps its seed and generator inside kolakoski().

Sources: Practice A Q8's memo_fun hint; syllabus memoization and environment model; Source Academy SICP 3.5.1.

G17 | Kolakoski: disentangle values, runs and cursors

A run is a maximal consecutive block of equal values. For [1,2,2,1,1,2], the run symbols are [1,2,1,2] and their lengths are [1,2,2,1]. The Kolakoski sequence uses its own sequence values as those run lengths, with symbols alternating between 1 and 2, starting with 1.

Do not confuse an element index with a run index. A run consumes one length entry but may emit one or two output entries. These advance at different speeds. This distinction is the central idea behind many decoding and compression problems, even when the sequence is unfamiliar.

A finite seed breaks the apparent circularity

The first run is one 1. The next is two 2s. Thus the known prefix is [1,2,2]. Runs 0 and 1 are complete, so the next unread run length is at sequence index 2. Its value is already 2. The next symbol must be 1.

Prefix before extension	Read index	Length read	Symbol appended	Prefix afterward
[1,2,2]	2	2	1	[1,2,2,1,1]
[1,2,2,1,1]	3	1	2	[1,2,2,1,1,2]
[1,2,2,1,1,2]	4	1	1	[1,2,2,1,1,2,1]
[1,2,2,1,1,2,1]	5	2	2	[1,2,2,1,1,2,1,2,2]

First understand the conceptual generation state

This is a mathematical trace, not an instruction to resize an indexed list. **prefix** denotes valid sequence elements generated so far. **read** points to the next sequence element to use as a run length. **symbol** is the next run's value. **i** is the output position requested by the stream consumer. The first three describe generation; i describes consumption.

When i is already inside prefix, emit prefix[i]. When i reaches the end, extend prefix by the next complete run, then emit. Since run lengths are only 1 or 2, one extension costs constant work. Move read one step per run and toggle symbol using 3-symbol.

Why the needed length is available

At the seed, len(prefix)-read is 1. Appending a run of length L in {1,2} and then advancing read changes this gap by L-1, which is never negative. Therefore the unread length entry remains available whenever another run must be generated. This invariant, rather than guesswork about the first few terms, justifies indefinite progress.

Source focus: Practice A Q8. G18 realizes this dependency with memoized stream cursors, so it needs neither indexed-list append nor a fixed maximum output size.

G18 | Kolakoski with a seed and memoized stream cursors

Use the memo_fun definition from G16. Seed three nodes, then expand run lengths read from the very same memoized sequence. The reader starts at element 2, whose value is already 2; the next run symbol is 1. A run length cursor advances once per run, while the output advances once per emitted element.

```
def kolakoski():
    def runs(lengths, symbol):
        return emit(symbol, head(lengths), lengths)
    def emit(symbol, left, lengths):
        def later():
            if left > 1:
                return emit(symbol, left - 1, lengths)
            return runs(stream_tail(lengths), 3 - symbol)
        return pair(symbol, memo_fun(later))
    def after_seed():
        return runs(stream_tail(stream_tail(sequence)), 1)
    sequence = pair(1, memo_fun(lambda: pair(2, memo_fun(
        lambda: pair(2, memo_fun(after_seed))))))
    return sequence
```

State and productivity

emit(symbol,left,lengths) emits one symbol now. left includes that emitted element. If more copies remain, its delayed tail keeps the same lengths cursor. Otherwise the delayed tail advances lengths once and toggles the symbol with 3-symbol. after_seed runs only after sequence has been assigned and the first three nodes exist.

The gap argument in G17 ensures the next required length is already available. Memoized seed tails are essential: later readers must reach the same cached nodes, not rebuild unfinished dependencies. The self-reference has a seed and a dependency order; merely wrapping an unproductive cycle in memo_fun would not fix it.

Cost and checks

Construction is O(1). Every new element and run transition performs constant stream work, so a single advancing cursor produces n values in O(n) node operations under unit-cost arithmetic. Retaining the original head retains O(n) generated nodes. No indexed list grows.

Check the supplied 20-element prefix, then generate 1000 elements. Group that prefix into runs and compare all completed run lengths with the initial sequence values. Omit the final observed run because it may be cut off. Force a tail twice and compare identity; interleave two independent instances to detect accidental shared state.

Transfer: separate a known seed, input cursors, remaining output count, and delayed next state. The dojo's smooth-number stream uses different rules but the same discipline. Source focus: Practice A Q8.

G19 | Destructive updates: preserve links before changing them

A destructive function reuses existing mutable nodes. The critical resource is the graph of links, not merely the sequence of values. Once you overwrite a tail, the old suffix may become inaccessible unless another variable still refers to it.

```
def d_reverse(xs):
    previous = None
    while not is_none(xs):
        nxt = tail(xs)          # preserve unprocessed suffix
        set_tail(xs, previous)  # reverse this edge
        previous = xs
        xs = nxt
    return previous
```

The invariant is that previous heads a reversed chain of the nodes already processed, while xs heads the remaining original-order chain. They are disjoint and together contain every original node. The first redirected tail points to None, which becomes the final terminator. No new pair is created.

Cutting a chain

To split after the k-th node, first save second = tail(last_of_first), then set_tail(last_of_first,None). Merely remembering two head variables does not split the structure: if the link remains, the first half still reaches the second half. That overlap can cause destructive recursive calls to corrupt each other.

Building by reuse

For destructive filtering or partitioning, retain first and last references for each output chain. Save the incoming next pointer, detach the current node, then append it to the appropriate chain. A dummy/sentinel node simplifies some algorithms but is illegal if the task says no new pairs. Handle the first node explicitly instead.

What aliases observe afterward

If xs originally points to the first node, reversing changes that node into the last node. The caller's xs still names that original node, not automatically the new head. The function must return the new head and the caller must use it. This is the same principle as C++ pointer reassignment versus changing the object a pointer targets, although Python names are not explicit pointer variables.

The common assumption for destructive merge is that inputs are finite, acyclic and share no nodes. If two input lists share a suffix, naive relinking may create a cycle or duplicate visits. The supplied Practice A examples use independent lists; the dojo explicitly requires disjoint inputs rather than silently depending on that property.

Source focus: Practice A Q9 and syllabus mutable data. Check identities and final tails, not just printed heads.

G20 | Destructive merge and merge sort

For two ascending inputs, the next result node must be the smaller available head. Keep that node, recursively merge the remainder, and attach the returned chain as its new tail.

```
def d_merge(xs, ys):
    if is_none(xs): return ys
    if is_none(ys): return xs
    if head(xs) <= head(ys):
        set_tail(xs, d_merge(tail(xs), ys))
        return xs
    set_tail(ys, d_merge(xs, tail(ys)))
    return ys
```

Choosing xs on equality preserves its position before equal values from ys. Returning the chosen node preserves its identity. Writing return pair(head(xs),...) would produce the right values but violate the no-new-pairs rule. Swapping head values is also wrong when only tail changes are permitted.

Sorting adds a split, not a new merge idea

```
def d_merge_sort(xs):
    if is_none(xs) or is_none(tail(xs)):
        return xs
    slow = xs
    fast = tail(xs)
    while not is_none(fast) and not is_none(tail(fast)):
        slow = tail(slow)
        fast = tail(tail(fast))
    second = tail(slow)
    set_tail(slow, None)
    return d_merge(d_merge_sort(xs), d_merge_sort(second))
```

The slow cursor advances once while fast advances twice, locating a split with roughly equal sizes. The empty/singleton base means neither recursive half can get stuck repeating the same problem. The explicit cut creates disjoint halves. Sorting each and merging produces the entire ordered list.

Reason about resource use honestly

Merging n+m nodes takes O(n+m) time. The merge calls are NOT tail calls: set_tail still needs their returned results. They can keep O(n+m) frames despite proper tail-call support and despite allocating no pairs. Balanced merge sort takes O(n log n) time; its recursive split depth is O(log n), but the recursive merge above can dominate auxiliary stack with O(n). An iterative merge would reduce that contribution, giving O(log n) stack for the split recursion. "Destructive" does not by itself imply O(1) auxiliary space.

Sorted inputs plus a merge produce a sorted result; merge alone does not sort arbitrary inputs. To prove stability of merge sort, use stable recursive halves and a tie rule that takes from the earlier left half first.

Source focus: Practice A Q9; syllabus sorting and searching. Test duplicate-node identity order as well as numeric order.

G21 | Complexity from actual operations

Big-O describes an asymptotic upper bound under a stated cost model. Theta describes a matching growth order. Always state the variable: route length n, dimensions R and C, token count t, or number of requested stream outputs n. Input shape and representation matter.

Operation / algorithm	Typical cost under this guide's model
head, tail, set_tail, pair	O(1)
Indexed-list access/update, len	O(1)
Preallocated stack push/pop by top index	O(1)
Allocate k indexed-list slots	O(k)
Linked append(xs,ys)	O(length(xs))
Linked-list index / length	O(n) traversal
Binary search in an indexed sorted array	O(log(n+1))
Linear traversal / merge	O(n) / O(n+m)
Insertion sort, worst case	O(n^2)
Balanced merge sort	O(n log n)
DFS enumerating simple maze paths	Exponential worst case
First n stream values	Depends on forcing, caching and traversal

Recurrence patterns

T(n)=T(n-1)+O(1) is linear. T(n)=T(n-1)+O(n) is quadratic. T(n)=2T(n/2)+O(n) is n log n. Two recursive calls do not automatically imply exponential work: merge sort splits into disjoint halves. Naive Fibonacci repeats overlapping subproblems; memoization can collapse those duplicates when the key fully determines the answer.

Binary search uses a shrinking interval

For lower_bound, maintain [lo,hi) as the unresolved interval. Everything before lo is below target; everything at or after hi meets or exceeds it. Initially lo=0, hi=len(a). If a[mid]<target, set lo=mid+1; otherwise set hi=mid. When lo==hi, that boundary is the answer, possibly len(a). No slicing is needed.

Space has several categories

Distinguish existing input, newly allocated output, auxiliary working data and recursive stack. A fresh n-node linked list necessarily uses O(n) output. An iterative destructive reverse uses O(1) auxiliary space. A memoized stream may hold onto its entire forced prefix. A proper tail call in §3 adds no pending control frame; non-tail recursion and retained closures still use space. Huge integers make arithmetic cost grow beyond the unit-cost model.

The practical target is not to memorize a table blindly, but to count how often each expensive operation occurs. A repeated copy, linked-length traversal or restarting stream_ref can turn an apparently linear algorithm quadratic.

Sources: syllabus orders of growth, searching, sorting and memoization; operation models used explicitly in Practice A Q7-Q8. Costs refer to §3 indexed lists and pair-based linked lists.

G22 | Trees and continuation-passing style

A recursive data definition tells you the cases your code must handle. For this guide's illustrative numeric tree, a tree is None, a numeric leaf, or a linked list of trees. This is an explicitly chosen representation; use the actual constructor and selector contract when your question supplies a different one.

```
def tree_sum(tree):
    if is_none(tree): return 0
    if not is_pair(tree): return tree
    return tree_sum(head(tree)) + tree_sum(tail(tree))
```

The head can itself be a nested tree, while the tail contains the remaining siblings. Processing only tails would skip subtrees. For a pair tree, the recursion depth can reflect a long sibling chain as well as visible nesting; do not automatically call it logarithmic.

A continuation describes the rest of the computation

A direct recursive sum squares the head, waits for the suffix sum, then adds. CPS packages that pending work in a function k. The contract becomes "compute the sum, then return what k does with it."

```
def sum_squares_cps(xs, k):
    if is_none(xs):
        return k(0)
    value = head(xs)
    return sum_squares_cps(
        tail(xs), lambda rest_sum: k(value * value + rest_sum))
```

With input [2,3] and initial k(v)=v+1, the base calls a chain of continuations: 0 becomes 9, then 13, then 14. A continuation can also return a list or format a value; the function must propagate that result rather than assuming it is numeric.

Why returning matters twice

The recursive call's result must be returned, and the continuation must return the old continuation's result. Accidentally calling k(...) as a statement and then falling off the function returns None. In Python a lambda automatically returns its expression; a def body needs an explicit return.

CPS is a control representation

It makes pending work explicit and helps reason about evaluation order, early exits and interpreter control flow. In §3 the descent and continuation calls shown are tail calls, so control-stack depth is O(1). However, descent allocates O(n) closures representing pending additions. Constant control space is not constant total auxiliary memory. If the problem only needs an iterative sum, a loop avoids that closure chain; use CPS when that representation or learning objective is requested.

For functional tree mapping, replace numeric leaves with f(leaf), preserve None, and construct new pairs from recursively transformed head and tail. That combines higher-order functions with structural recursion and preserves nesting.

Source focus: syllabus Tree Processing and CPS; Function Objects and Loops. This chapter supplies general concepts because no detailed course examples were attached.

G23 | Symbolic expressions and environments

A symbolic program can be represented as ordinary data. The evaluator reads the data's structure and determines its value. To avoid inventing the course evaluator, use this small explicitly defined language as an illustration:

Syntax	Meaning
['num',n]	Number n
['var',name]	Look up name in the current environment
['add',a,b]	Evaluate a and b, then add
['mul',a,b]	Evaluate a and b, then multiply
['let',name,rhs,body]	Evaluate rhs, bind name locally, evaluate body

An environment is a chain of bindings, newest first. A binding record is pair(name,value). The environment itself is a linked list of those records. Lookup scans from newest to oldest, so an inner binding shadows an outer binding with the same name.

```
# Example environment extension, without changing the old environment:
new_env = pair(pair(name, value), old_env)
```

Because a let's right-hand side is evaluated in the old environment, let x = x+3 inside an outer x=2 gets the value 5. Only the body sees the new x binding. Evaluating rhs in the extended environment can change the language's meaning or refer to an uninitialized value.

Connect to function application

For a full lexically scoped evaluator, a function value typically carries parameters, a body and the environment where it was created. A call evaluates its argument expressions, binds parameter names to those values in a new frame linked to the function's creation environment, then evaluates the body there. The caller's local environment is not automatically the function's parent environment.

Primitive application and compound-function application are different cases: the first delegates to an underlying operation, while the second evaluates a body with a new frame. A conditional must evaluate the selected branch, not eagerly evaluate both branches before choosing. Otherwise an unused branch could divide by zero, mutate state or loop forever.

Substitution versus environment models

Substitution is a useful model for many pure function calls. With assignment and shared mutable objects, names and object identities need persistent locations/bindings, so an environment model becomes essential. A box-and-pointer diagram records which references reach which objects; an environment diagram additionally records name bindings and function creation environments.

Source focus: syllabus symbolic processing, Environment Model and Metacircular Evaluator. The exact tags, helper names, return handling and evaluation rules of your course evaluator must come from its supplied materials; the screenshots only establish the topic.

G24 | Cover the wider syllabus without guessing APIs

The final screenshots name more topics than Practice A tests directly. Preparing for surprises means understanding their transferable ideas while distinguishing known topic names from missing course-specific details.

Syllabus area	Transferable concept	Preparation connection
Elements of programming	Expressions, definitions, conditionals, composition	G03 and contracts
Runology	Values represent images; functions transform/combine them	Higher-order composition
Curves and sound	Functions can represent time/parameter-dependent objects	Closures and transformations
Data abstraction	Constructors/selectors hide representation	G08 and pair records
Tree/symbolic processing	Data shape determines recursive cases	G22-G23
Scope/environment model	Bindings, sharing, lexical capture	G04, G07, G16
Searching/sorting	Invariants, boundaries, ties and resource costs	G20-G21
Memoization	Cache only by a complete subproblem state	G13, G16
Streams I/II	Deferred computation, infinite data and sharing	G14-G18
Metacircular evaluator	Syntax, environments and application	G23

Function-valued data

A curve can be modeled abstractly as a function from a parameter t to a point; a sound can be modeled using a time-varying signal. A transformation may therefore return another function, such as lambda t: transform(original(t)). The exact argument order, durations and constructor names are not given in your screenshots. Learn to read the provided contract rather than importing guessed library calls.

Abstraction is a practical debugging tool

When an unfamiliar problem introduces a "route", "run", "environment" or "signal", first write its representation separately from its operations. A route might be a linked list of integers; a run might be pair(value,count); a stream of runs is a pair whose head is that record and whose tail is a thunk. Each layer has a different selector purpose. Naming intermediate values makes these layers visible.

Sound inference versus invented certainty

The nine practice questions strongly justify emphasizing mutable lists, loops, streams and search. The syllabus justifies keeping earlier topics available as supporting knowledge. The current Python §3 reference establishes the base language, while a particular assessment can impose further restrictions or supply extra helpers. This guide therefore includes self-contained reference implementations and labels supplementary algorithms rather than claiming they are official templates.

Parsing and underlying-evaluator helpers listed under Python §4 are not automatically §3 primitives. The miniature evaluator in G23/D35 uses ordinary §3 data and functions with an explicitly defined syntax.

Sources: attached syllabus screenshots, weeks 1-6 and 7-11. No additional lecture content was available in the attachments.

G25 | Test the contract, not just the sample

A good test distinguishes a likely wrong algorithm from a correct one. Copying the problem's examples is a start, but hidden tests usually target boundaries, wrong assumptions and side effects.

Failure mode	A useful distinguishing test
Missed empty/base case	None, singleton, one-cell goal
Adjacent traversal skips links	At least four unequal values
Reversed matrix coordinates	Asymmetric directed matrix
Row/column confusion	Nonsquare matrix and 1-by-C/R-by-1
Shared matrix rows	Mutate one output row; inspect another
Reversed postfix operands	Subtraction and division
Wrong accumulator	Negative differences and repeated values
Missing restoration	Compare maze before/after; call twice
Illegal pair allocation	Compare exact node sets before/after
Lost or duplicated nodes	Identity list, cycle detection, length
Unstable merge	Equal values with distinct original nodes
Eager stream work	Construction and bounded forcing counters
Recomputed shared tails	Force one tail twice; compare identity
Bad cache sentinel	A legitimate cached 0 or None result

Properties give more coverage than memorized outputs

Delta decode(encode(xs)) should reproduce xs structurally and neither stage should modify xs. Transposing twice should reproduce the original numeric matrix. Destructive reverse twice should restore the original node order and links. Merge should preserve the multiset of values and exact union of input nodes. A scan's adjacent output differences should reproduce later input contributions. These are relationships between operations, called metamorphic properties.

For nonmutating linked functions, record each original node's identity, head and tail before the call. For destructive functions, check the returned chain contains exactly the required original identities and ends at None. Numerical equality alone cannot detect replacement nodes or cycles.

Read a failing trace with a specific hypothesis

An error near head may mean a thunk or atomic value was mistaken for a pair. An unexpected None often means a missing return. An invalid index suggests a wrong boundary; reject negative coordinates before accessing a grid. Nontermination suggests an unchanged recursive argument, an unintended cycle or an eagerly forced infinite stream. A parser error on otherwise familiar Python often signals unsupported syntax. Distinguish syntax, representation and algorithm failures before editing.

Before changing several lines, write what each variable means at the failing step. Trace the smallest counterexample. Fix the violated invariant rather than patching that one observed value. Use the supplied §3 checks and print(...) to inspect a small trace. Finite tests cannot establish productivity of every stream or correctness on every maze.

Source focus: contracts across Practice A Q1-Q9. The companion practice kit checks values and important structural properties; passing it is evidence, not a proof of all possible inputs.

G26 | Use the pack to become independent on Practice B

The long guide teaches why a pattern works. The two-sided A4 sheet is a retrieval aid. The dojo turns recognition into implementation. The separate solution book provides two staged hints, an explanation, complexity and reference code for each exercise; keep it closed during an initial attempt.

Browser-only workflow

Open the website, choose a dojo problem, and write in its editor. Copy the runnable program, open Source Academy, select Python §3, paste and run. The copied program includes helpers and checks without revealing the reference solution. Successful checks print a completion message. No Python installation or VS Code is needed. Drafts stay in the current browser tab session; download your code if you want a durable copy.

A progression that tests understanding

1. Read G02-G08, then attempt dojo D01-D06. Write the meaning of each helper argument before writing its body.

2. Add G09-G10 and attempt D07-D12. Draw the data shape and test a nonsquare or noncommutative example.

3. Read G19-G21 and attempt D13-D18. Draw node identities and the link that each update changes.

4. Read G11-G13 and attempt D19-D24. Explain what visited means and which changes must be undone.

5. Read G14-G18 and attempt D25-D30. Mark which expressions run now and which run when forced.

6. Read G22-G24 and attempt D31-D36. These add syllabus transfer and combine several patterns.

The stages increase in accumulated concepts; individual difficulty depends on your strengths. The final stage mixes foundations with harder integration rather than pretending every later short task must be harder than every earlier one.

A hint ladder

First retry with only the relevant guide page. Next consult Hint 1, which identifies the invariant or representation. Only then use Hint 2, which suggests the structural step. Read the full code after a genuine implementation attempt. Later, close the solution and reproduce the algorithm from the contract. Merely recognizing a familiar answer is a weaker skill than rebuilding it.

Three mixed rehearsal sets

Core transfer: D03, D05, D10, D12, D19, D25, D16. **Changed rules:** D09, D11, D18, D21, D24, D27, D31. **Stretch synthesis:** D22, D23, D29, D30, D32, D35, D36. Choose a set without reading solutions, use only the guide and sheet, and compare afterward. These are original rehearsal selections, not claims about Practice B's duration, scoring or permitted references. Official conditions take precedence.

A short error log worth keeping

For each miss, record the violated idea rather than only the line fixed: representation, base case, progress, state update, operand order, aliasing, restoration, laziness, complexity or identity constraint. Rewrite one sentence explaining why the correction works. Reattempt a changed version after a gap so you test transfer rather than short-term copying.

On Practice B, begin by classifying the representation and contract, then design state and base cases from scratch. Use the sheet to retrieve an idea, not to choose a template merely because the story sounds familiar. If an unfamiliar term appears, translate it into data and permitted operations before coding.

Source basis: your stated Practice B plan, supplied Practice A questions and syllabus. The training design and dojo are original.